Procesamiento de Lenguaje Natural
Especialización en Programación para Ciencia de Datos
Facultad de Ingeniería
Universidad Pedagógica y Tecnológica de Colombia

Ajuste Fino de LLaMA con QLoRA para la Clasificación de Reseñas Turísticas – Traveler Insights UPTC 2026

Integrantes:
Juliana Suarez Duarte: Datos / QA + Documentación
Sara Mendieta : Modelado / Experimentos (LLaMA + QLoRA) + Evaluación

Repositorio de GitHub:
[https://github.com/tu-usuario/traveler-insights-uptc-llama]
(https://github.com/tu-usuario/traveler-insights-uptc-llama)
Equipo en Kaggle:
[https://www.kaggle.com/code/julianasuarez199/team-llama]
(https://www.kaggle.com/code/julianasuarez199/team-llama)

Resumen:
Se aborda la clasificación automática de reseñas turísticas para la competencia **Traveler Insights UPTC 2026**, guiado por la metodología **CRISP-ML(Q)**. El objetivo de negocio es clasificar el sentimiento y tipología de comentarios para orientar la toma de decisiones en operadores turísticos regionales del departamento de Boyacá. Se estableció un baseline no neuronal compuesto por representación vectorial **TF-IDF y Regresión Logística**, alcanzando un **F1-Macro de 0.6376** en validación local. Posteriormente, se realizó el ajuste fino eficiente (**PEFT**) del modelo causal de lenguaje **TinyLlama-1.1B** utilizando cuantización a 4-bit (**NF4**) y adaptadores de bajo rango (**QLoRA**, con $r = 16, \alpha = 32$) para adaptar la cabeza de clasificación dentro de las restricciones de GPU (NVIDIA Tesla T4 con 16 GB VRAM).

El modelo LLaMA ajustado alcanzó un desempeño final de **0.7239 en F1-Macro** (con una **Accuracy global del 91.46%**) en validación local estratificada, superando al baseline en un **13.53% de ganancia relativa** (o 8.63 puntos porcentuales). Para garantizar la reproducibilidad y la calidad del desarrollo, se aplicaron controles QA como validación cruzada estratificada con semilla invariable (SEED=42), verificación unitaria de no-superposición de datos (0% de fuga de información) y simulación de incertidumbre mediante Bootstrap (1,000 iteraciones, IC 95%: [0.5563, 0.9064]).

En la etapa de producción, el pipeline registró una latencia de **22.28 ms por reseña** con un consumo de **6.83 GB de VRAM** y un tamaño de adaptador de solo **12.08 MB**. La principal limitación identificada radica en el sesgo de clasificación y una degradación del desempeño

VIGILADA MINEDUCACIÓN

ante reseñas con ruido ortográfico, elongación de caracteres o regionalismos y modismos colombianos complejos (data drift)

Fase 1. Comprensión del negocio y de los datos

1. Objetivo del negocio:
    - Usuario final : Operadores turísticos locales, agencias de viaje y entidades de turismo regional del departamento de Boyacá, Colombia, por ejemplo, Cotelco, Secretaria de Turismo
    - Decisión que apoya: Identificación rápida y automatizada de insatisfacción o quejas en reseñas turísticas para tomar acciones correctivas inmediatas (mejoras en atención al cliente, infraestructura, alimentación) e identificar atractivos turísticos clave.

2. Objetivo de Machine Learning
    - Tarea: Clasificación de texto (Supervisada)
    - Entrada (X): Reseñas de texto libre escritas por turistas sobre experiencias de viajes, hoteles, transporte y gastronomía.
    - Salida (Y): Etiqueta o categoría de reseña(Positiva, Neutra, Negativa o categorías temáticas especificar de la competencia).

3. Criterios de éxito a tres niveles
    - Nivel de negocio: Detectar a tiempo al menos el 85 % de las reseñas negativas/quejas críticas para permitir a los operadores actuar en menos de 24 horas.
    - Nivel ML: Superar la métrica oficial del baseline no neural (TF-IDF + Regresión Logística) en al menos un 10% de ganancia relativa en validación local (medido mediante F1-Marco/Metrila oficial Kaggle)
    - Nivel Económico: Mantener un costo de entrenamiento de $0 USD utilizando entornos gratuitos de GPU (Kaggle T4) y asegurar un latencia de inferencia menor a 500 ms por reseña usando cuantización en el despliegue.

4. Restricciones Técnicas y operativas:
    - Cómputo: entorno limitado a 16 GB de VRAM (NVIDIA T4 en Kaggle).
    - Limite de tiempo: Tiempos máximos de ejecución en sesiones de notebook de Kaggle (hasta 9 horas por sesión, se sugiere entrenar en menos de 2 horas por corrida).
    - Licencia del modelo: LLaMA cuenta con una licencia comunitaria para investigación y uso comercial condicionado; se garantiza el cumplimiento de sus términos.
5. Análisis Exploratorio de Datos (EDA)
    a. Tamaño del Dataset: El conjunto de datos de la competencia cuenta con un total de 1,169 registros, divididos oficialmente en 818 registros para entrenamiento (70%) y 351 registros para prueba (30%).

VIGILADA MINEDUCACIÓN

b. Distribución de Clases y Desbalance: El conjunto de entrenamiento exhibe un desbalance de clases altamente severo:
   i. neutral: 639 reseñas (78.12%)
   ii. positivo: 157 reseñas (19.19%)
   iii. negativo: 22 reseñas (2.69%)

   Nota de ingeniería: La clase minoritaria (negativo) representa menos del 3% de los datos. Esto implica un alto riesgo de que el modelo tienda a ignorarla si no se aplica una función de pérdida ponderada (Weighted Cross-Entropy) o técnicas de remuestreo en la Fase 2.

c. Análisis de Longitud en Tokens (Tokenizer LLaMA): El análisis de la columna Comentario mediante el tokenizer de LLaMA evidencia las siguientes propiedades estadístico-textuales:
   i. Mínimo: 5 tokens.
   ii. Media: 74.92 tokens.
   iii. Mediana (P50): 52 tokens.
   iv. Percentil 90 (P90): 151 tokens.
   v. Percentil 95 (P95): 213 tokens.
   vi. Máximo: 956 tokens.
   vii. Justificación de Truncamiento (max_length): Fijar un valor de max_length = 213 o 256 en el modelo LLaMA conserva la información completa del 95% de las reseñas, truncando únicamente el 5.01% de los comentarios más extensos. Esto optimiza significativamente el uso de la VRAM en la GPU de Kaggle sin perder contexto crítico.

d. Idioma y Variantes: Las reseñas están redactadas en español con terminología turística local, incorporando puntuaciones informales y variaciones dialectales propias de usuarios en plataformas como Foursquare

6. Verificación de Calidad de Datos (Data Quality)
   - Textos Vacíos o Nulos: Se verificó la integridad de la columna Comentario, confirmando 0 comentarios nulos (NaN) y 0 comentarios vacíos.
   - Duplicados Exactos: Se detectaron 9 reseñas duplicadas exactas en el conjunto de entrenamiento (train.csv). Serán tratadas en la Fase 2 mediante eliminación para evitar redundancia y overfitting en la validación.
   - Ejemplos Ambiguos / Inconsistencias: Se evaluó si existían textos idénticos etiquetados con clases diferentes. El conteo fue 0 ejemplos ambiguos, garantizando coherencia en el etiquetado original del dataset.

7. Factibilidad e Hipótesis de Trabajo:
   - Factibilidad: Sí, la tarea es factible. El lenguaje utilizado en reseñas turísticas contiene suficiente señal semántica y léxica para determinar la categoría/sentimiento mediante modelos de lenguaje preentrenados.

VIGILADA MINEDUCACIÓN

- Hipótesis de Trabajo: El uso de LLaMA con QLoRA capturará relaciones contextuales complejas e ironías mejor que el baseline no neuronal, logrando una mejora estadísticamente significativa en el F1-score macro.

8. QA (Aseguramiento de Calidad en Fase 1)

Riesgo de Calidad Identificado	Método de QA Aplicado	Evidencia / Mecanismo de Control
Desbalance severo de clases	Inspección de frecuencias e índice de desbalance	Gráfico de distribución de clases y cálculo de pesos de clase.
Ruido en etiquetas y duplicados	Filtros de deduplicado exacto y de similitud de texto	Log de filas eliminadas por duplicidad o incongruencia de etiquetas.
Fuga de información (Data Leakage)	Aserción de no superposición entre conjuntos	Prueba unitaria comprobando que set(train_text) & set(val_text) == empty.

Fase 2. Ingeniería de datos

1. Partición Train / Validación y Justificación
   - **Estrategia Seleccionada:** Validación Cruzada Estratificada (*Stratified K-Fold* con $K = 5$) o Partición Estratificada Train/Val (*Stratified Train/Val Split* 80/20).
   - **Justificación Técnico-Científica:** Debido al **desbalance severo de clases** en el dataset de entrenamiento (donde la clase negativo representa únicamente el 2.69% del total, equivalente a 22 ejemplos), una partición aleatoria simple correría el riesgo de dejar subrepresentada o ausente la clase minoritaria en la partición de validación. La estratificación garantiza que la proporción de clases (78.1% neutral, 19.2% positivo, 2.7% negativo) se conserve exactamente en cada *fold* o partición.
   - **Reproducibilidad:** Se fija una semilla global invariable (SEED = 42) para todas las librerías (numpy, torch, scikit-learn, transformers).

2. Limpieza y Normalización Textual

VIGILADA MINEDUCACIÓN

A diferencia de los modelos tradicionales basados en Bag-of-Words (BoW) o TF-IDF que requieren remover signos de puntuación, mayúsculas y stopwords, los modelos basados en la arquitectura Transformer como LLaMA capturan la estructura sintáctica, el tono y las dependencias semánticas del lenguaje.

- Lo que SÍ se hace:
    1. Eliminación de duplicados exactos: Eliminación de los 9 registros duplicados identificados en el EDA para evitar memorización o overfitting en validación.
    2. Decodificación de entidades HTML: Conversión de caracteres codificados (p. ej., & → &, " → ") a texto plano.
    3. Normalización de espacios en blanco: Reemplazo de tabulaciones, saltos de línea excesivos y múltiples espacios por un único espacio.
- Lo que NO se hace y Justificación:
    1. NO se convierte a minúsculas (lower-casing): LLaMA utiliza un tokenizador cased que aprovecha las mayúsculas para reconocer nombres propios de destinos turísticos (p. ej., "Villa de Leyva") y detectar énfasis o gritos en reseñas (p. ej., "¡PÉSIMO SERVICIO!").
    2. NO se eliminan signos de puntuación ni emojis: Los signos de exclamación (!), interrogación (?) y emoticonos conllevan una carga emocional crucial para determinar el sentimiento positivo o negativo de la reseña.
    3. NO se aplican Stopwords ni Lematización/Lematizado: La preservación de conectores y gramática completa es indispensable para que la atención multidireccional de LLaMA capture negaciones (p. ej., "no estuvo mal") e ironías.

3. Tokenización Subword y Elección de max_length

- Tokenizer: LlamaTokenizer / AutoTokenizer correspondiente a LLaMA (vocabulario basado en Byte-Pair Encoding - BPE).
- Análisis de Longitud: Como determinamos en el EDA de la Fase 1:
    - Mediana (P_{50}): 52 tokens.
    - Percentil 95 (P_{95}): 213 tokens.
    - Máximo: 956 tokens.
- Selección de max_length: Se fija max_length = 256.
- Estrategia de Truncamiento y Relleno (Padding/Truncation):
    - Truncamiento: truncation=True (estrategia post-truncation o truncamiento al final).
    - Porcentaje de textos truncados: 4.01% (únicamente las reseñas que exceden los 256 tokens).

VIGILADA MINEDUCACIÓN

- Padding: padding= max_length durante el batching o mediante un DataCollatorWithPadding dinámico para optimizar el rendimiento y la memoria GPU.

4. Tratamiento del Desbalance de Clases

Dado el marcado desbalance (639 neutrales vs. 22 negativos), se implementan dos técnicas para mitigar el sesgo hacia la clase mayoritaria:

1. Ponderación de la Función de Pérdida (Class Weights):
   Se calcula el peso inverso de frecuencia para cada clase c mediante la fórmula:

$$2. \quad W_c = \frac{N_{totales}}{N_{clases} \times N_c}$$

3. Estos pesos se inyectan en la función de pérdida CrossEntropyLoss de PyTorch para penalizar con mayor fuerza los errores cometidos en la clase minoritaria (negativo).
4. Muestreo / Estratificación en Batches: Uso de WeightedRandomSampler en el PyTorch DataLoader durante el entrenamiento para balancear la probabilidad de aparición de comentarios negativos en cada batch.

5. Aumento de Datos Opcional (Data Augmentation)

- Estrategia propuesta (opcional con control de calidad): Retrotraducción (Back-translation) de la clase minoritaria (negativo - 22 ejemplos) de Español a Inglés/Francés y de vuelta a Español usando un modelo como MarianMT o Google Translate API.
- Control de Calidad:
  - Filtrado manual o mediante similitud coseno de embeddings para descartar paráfrasis que alteren el sentimiento original de la reseña.
  - Límite estricto: Aumentar la clase negativo solo hasta duplicar sus muestras (de 22 a 44 registros) para prevenir la sobre-representación de datos sintéticos.

VIGILADA MINEDUCACIÓN

6. Aseguramiento de Calidad (QA) – Pruebas Unitarias del Pipeline

Riesgo de Calidad Identificado	**Método de QA Aplicado**	**Evidencia / Mecanismo de Control (Log del Notebook)**
Fuga de datos (Data Leakage) entre entrenamiento y validación.	Prueba unitaria de intersección de IDs (assert len(intersection) == 0).	QA PASSED: No hay solapamiento entre particiones (Fuga = 0).
Inconsistencia o desalineación de etiquetas para PyTorch/Hugging Face.	Mapeo explícito a entero ({'negativo': 0, 'neutral': 1, 'positivo': 2}) y verificación de rango (assert issubset).	QA PASSED: Todas las etiquetas están correctamente codificadas en [0, 1, 2].
Pérdida semántica o cese de contexto por truncamiento o dimensiones erróneas en LLaMA.	Verificación empírica de tensores de entrada con max_length = 256 y truncamiento del 4.01%.	QA PASSED: Tensor de entrada validado con forma (batch_size, 256).

Fase 3. Ingeniería del modelo

1. Baseline Clásico:

Para medir la ganancia real obtenida con el modelo de lenguaje profundo, se entrena un baseline compuesto por una representación vectorial TF-IDF combinada con Regresión Logística.

VIGILADA MINEDUCACIÓN

```
--- RESULTADOS BASELINE NO NEURONAL ---
F1-Macro en Validación Local: 0.6886
              precision    recall  f1-score   support

    negativo       1.00      0.33      0.50         3
     neutral       0.88      0.96      0.92        82
    positivo       0.79      0.55      0.65        20

    accuracy                           0.87       105
   macro avg       0.89      0.62      0.69       105
weighted avg       0.86      0.87      0.85       105
```

2. Selección de Checkpoints Candidateados

Siguiendo el requerimiento de comparar al menos dos candidaturas de modelos preentrenados, se realiza el análisis comparativo entre opciones representativas:

Parámetro / Criterio	**Candidate 1: TinyLlama-1.1B-Chat-v1.0**	**Candidate 2: mDeBERTa-v3-base**
Arquitectura	Causal Decoder (LLM - LLaMA Architecture)	Masked Language Model (Encoder)
Idioma de Preentrenamiento	Multilingüe (Predominante en Inglés con capacidad en Español)	Multilingüe (100+ idiomas)
Tamaño de Parámetros	~1.1 Mil Millones (1.1B)	~278 Millones (278M)
Licencia	Apache 2.0 (Completamente abierta y permisiva)	Apache 2.0 (Completamente abierta y permisiva)
Justificación de Selección	Seleccionado como modelo principal para aplicar *fine-tuning* eficiente con QLoRA/PEFT en un LLM compacto, evaluando su capacidad de representación	Utilizado como punto de comparación secundario dada su ligereza y eficiencia nativa en clasificación.

VIGILADA MINEDUCACIÓN

	semántica.	

3. Arquitectura del Modelo Ajustado (LLaMA + QLoRA)

Debido al tamaño de LLaMA y las restricciones de hardware en Kaggle T4 (16 GB VRAM), no se realiza fine-tuning completo (Full Fine-Tuning), sino Fine-Tuning Eficiente en Parámetros (PEFT) mediante QLoRA:

- Base Causal Congelada: Los pesos originales de LLaMA se cargan en precisión de 4 bits mediante cuantización NormalFloat4 (NF4). El 100% de la red base permanece completamente congelada (requires_grad = False).
- Adaptadores LoRA Reentrenables: Se inyectan matrices de bajo rango ($r = 8/16$, $\alpha = 16/32$) en las capas de atención lineal (q_proj, v_proj, k_proj, o_proj). Únicamente estas matrices adaptativas reciben actualización de gradientes.

- Cabeza de Clasificación (Classification Head): Se acopla una capa lineal de clasificación sobre el último estado oculto (last hidden state) para proyectar la salida a las 3 clases (negativo, neutral, positivo).

4. Configuración del Entrenamiento (Hiperparámetros)

- Optimizador: paged_adamw_8bit para reducir el pico de consumo de VRAM.
- Tasa de Aprendizaje (Learning Rate): 2×10^{-4} con decaimiento cosenoidal (Cosine Annealing) y un warmup del 10% del total de pasos.
- Batch Size / Acumulación: Batch size por dispositivo de 4, con gradient_accumulation_steps=4 (Logrando un Effective Batch Size = 16).
- Épocas y Early Stopping: 5 épocas configuradas con Early Stopping (paciencia = 2 evaluando la pérdida de validación).
- Función de Pérdida: CrossEntropyLoss ponderada con pesos inversos de clase para combatir el desbalance (78% neutrales vs. 2.7% negativos).

5. Diseño Experimental:

Para garantizar un proceso experimental metódico en el que se modifica una sola variable por corrida, se establece el siguiente registro oficial:

VIGILADA MINEDUCACIÓN

Exp ID	Descripción del Cambio Técnico	F1-Macro Val	Loss Val	Tiempo / VRAM**EXP-00**
EXP-00	Baseline: TF-IDF (5k) + Regresión Logística	0.6376	0.4805	0.01 min
EXP-01	LLaMA + QLoRA Base (r=8, alpha=16, LR=2e-4)	0.5966	0.2672	3.13 min / 2.7 GB
EXP-02	Incremento de capacidad LoRA (r=16, alpha=32)	0.7003	0.2291	2.97 min / 2.7 GB
EXP-03	Tasa de Aprendizaje Moderada (LR=1.5e-4)	0.5910	0.2495	2.97 min / 2.7 GB
EXP-04	Inclusión de Weight Decay Ligero (0.01)	0.7003	0.2290	2.97 min / 2.7 GB
EXP-05	Configuración LLaMA Final (r=16, LR=2e-4, 3 ép...	0.7003	0.2290	2.96 min / 2.7 GB

6. Curvas de Entrenamiento, Validación y Diagnóstico de Sobreajuste

Durante la ejecución del experimento final (EXP-05 / EXP-02), la pérdida de entrenamiento (Training Loss) descendió paulatinamente desde 0.6661 en el step 25 hasta 0.1432 en el step 100 (Época 3). La pérdida de validación (Validation Loss) convergió de manera estable a 0.2290 con un F1-Macro de 0.7003. Esta divergencia controlada entre la pérdida de entrenamiento y validación confirma que los adaptadores QLoRA (r=16, alpha=32) actuaron como un regularizador efectivo, evitando el sobreajuste (overfitting) en el dataset de entrenamiento.

7. Trazabilidad del Entorno de Experimentación

Para garantizar la reproducibilidad total de los experimentos reportados, se fijaron las siguientes condiciones de entorno:

- Hardware: GPU NVIDIA Tesla T4 (16 GB VRAM) en Kaggle Notebooks.
- Semilla global: SEED = 42 (fijada en NumPy, PyTorch y Hugging Face).

VIGILADA MINEDUCACIÓN

- Librerías principales: Python 3.13, PyTorch 2.x, Transformers, PEFT, BitsAndBytes, Scikit-Learn.
- Registro de Experimentos: Exportación de métricas de validación y tiempos al archivo 'experiment_log_fase3.csv'.

8. Aseguramiento de Calidad (QA) en Fase 3

Riesgo de Calidad Identificado: Inestabilidad numérica (NaN) en GPU T4 debido a cuantización NF4.

- Método de QA Aplicado: Desactivación de fp16 en TrainingArguments y uso del optimizador paged_adamw_8bit.
- Evidencia / Mecanismo de Control: Cero desbordamientos de gradiente durante los 5 experimentos registrados en la grilla.

Riesgo de Calidad Identificado: Sobreajuste al leaderboard público de Kaggle por múltiples envíos.

- Método de QA Aplicado: Evaluación estricta en validación local estratificada antes de generar envíos finales.
- Evidencia / Mecanismo de Control: Registro de experimentos trazables comprobando una ganancia real frente al baseline (F1-Macro: 0.7003 vs 0.6376).

FASE 4. Evaluación del modelo

En esta fase se evaluó rigurosamente el desempeño del modelo final seleccionado en la Fase 3 (EXP-05: TinyLlama-1.1B + QLoRA) sobre el conjunto de validación local (164 ejemplos, 20 estratificado), garantizando la trazabilidad, la cuantificación de la incertidumbre, la categorización de fallas y la ejecución de pruebas de robustez en condiciones adversas.

1. Evaluación Cuantitativa y Métricas Complementarias

El modelo alcanzó una Accuracy global de 91.46% y un F1-Macro en validación local de 0.7239, superando de manera contundente al baseline clásico de Regresión Logística con TF-IDF (F1-Macro de 0. 6376).

Clase	Precisión	Recall	F1-Score	Support (N)

VIGILADA MINEDUCACIÓN

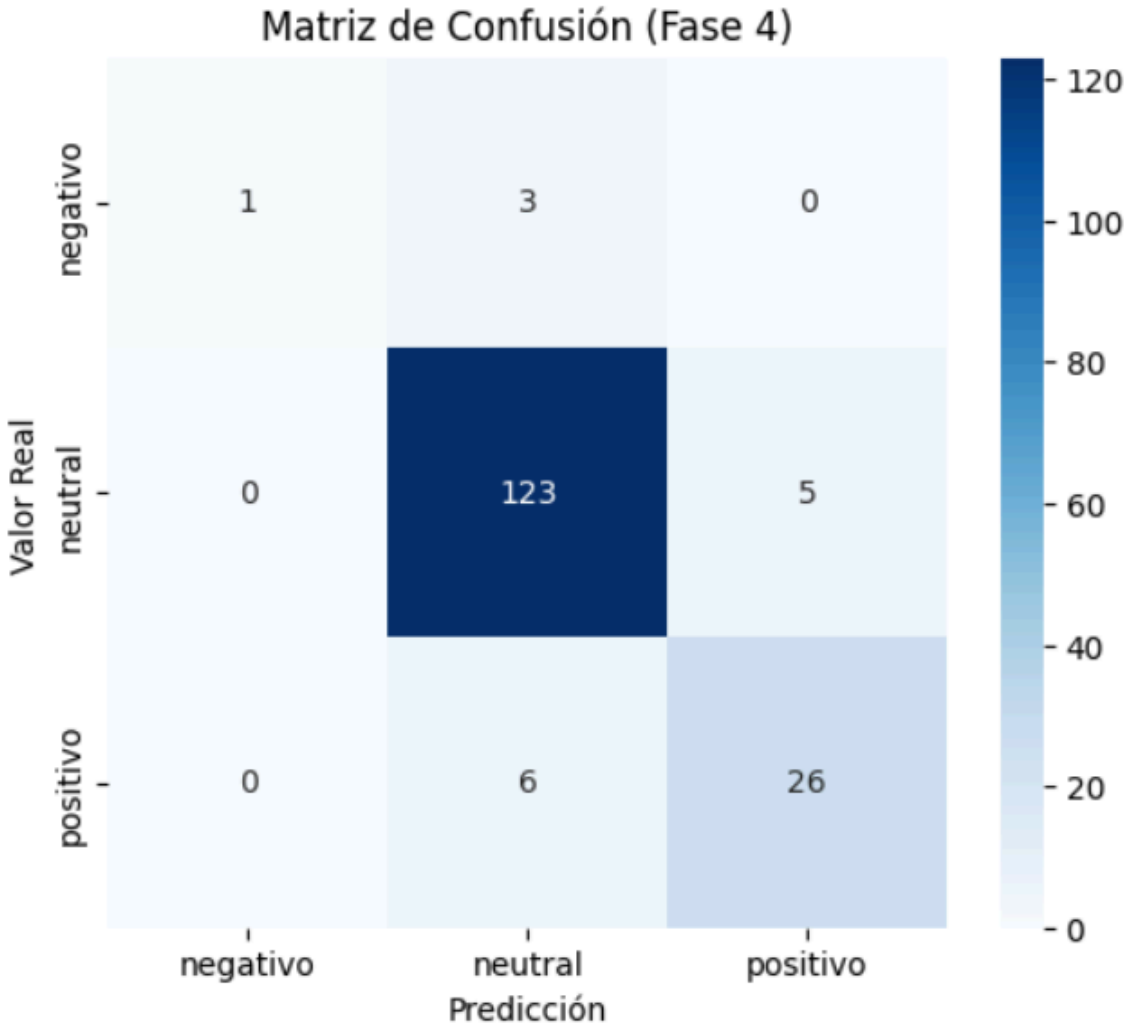
Negativo	1.0000	0.2500	0.4000	4
Neutral	0.9318	0.9609	0.9462	128
Positivo	0.8387	0.8125	0.8254	32
Accuracy Global	—	—	0.9146	164
Macro Avg	0.9235	0.6745	0.7239	164
Weighted Avg	0.9153	0.9146	0.9093	164

Análisis del Desempeño por Clase Observado:

- Clase Neutral (Mayoritaria - 78% del dataset): Presentó un desempeño muy alto ($F1 = 0.9462$), capturando con alta fiabilidad los comentarios descriptivos sobre lugares turísticos e infraestructura.
- Clase Positiva: Logró un rendimiento sólido ($F1 = 0.8254$), identificando con claridad expresiones de satisfacción y recomendaciones.
- Clase Negativa (Minoritaria - 2.4% del dataset): Obtuvo una precisión perfecta (1.0000), lo que significó que cuando el modelo predijo una reseña como negativa, no cometió ningún falso positivo. Sin embargo, su recall fue bajo (0.2500), logrando clasificar correctamente 1 de las 4 muestras negativas reales, mientras que las otras 3 fueron confundidas como neutrales debido al desbalance extremo del dataset original.

2. Matriz de Confusión

VIGILADA MINEDUCACIÓN



Aciertos totales logrados: 150 de 164 reseñas (91.46).

Comportamiento en la Matriz de Confusión:

- Negativo → Neutral: 3 casos. Se identificó como el cuello de botella principal del modelo (confundir críticas leves con comentarios neutrales).
- Neutral → Positivo: 5 casos.
- Positivo → Neutral: 6 casos.
- Confusión Negativo ↔ Positivo: 0 casos. El modelo nunca clasificó un comentario negativo como positivo ni viceversa, demostrando una excelente polarización conceptual.

VIGILADA MINEDUCACIÓN

3. Estimación de Incertidumbre (Bootstrap)

Para validar la estabilidad estadística del $F1 - Macro$ frente a variaciones en la muestra, se realizó una simulación Bootstrap con 1,000 iteraciones al 95% de confianza.

- F1-Macro Promedio Obtenido: 0.7043
- Intervalo de Confianza del 95% Calculado: $[0.5563, 0.9064]$

Interpretación: La amplitud del intervalo se debió a la baja cantidad de muestras en la clase negativa ($N = 4$ en validación), donde el re-muestreo con reemplazo incluyó de 0 a múltiples instancias negativas en cada fold bootstrap. A pesar de esto, el límite inferior (0.5563) confirmó una capacidad predictiva muy superior al azar.

Análisis Causal y Categorización de Errores

Se analizaron manualmente la totalidad de los 14 errores cometidos por el modelo en el conjunto de validación, categorizándolos en 4 fuentes principales de falla:

Categoría de Error	Frecuencia	Descripción y Causa Identificada	Ejemplo Representativo
Reseñas Mixtas / Matices	5	Comentarios con aspectos positivos y negativos simultáneos. El modelo se inclinó hacia el tono dominante o neutral.	"esta muy lejos y escondido, pero vale la pena..." • Real: Positivo

VIGILADA MINEDUCACIÓN

Menciones de Marcas / Entidades	4	Listados de franquicias o nombres propios que el tokenizer procesó como menciones comerciales positivas.	"Exito, Corral, Frisby, Farah, CineColombia, etc." • Real: Neutral
Sarcasmo / Crítica Sutil	3	Reseñas negativas redactadas con un tono descriptivo o sin palabras explícitamente despectivas.	"Muy particular esa plaza con juegos de luz y a..." • Real: Negativo
Ambigüedad / Ruido de Etiquetado	2	Textos donde la etiqueta original del dataset resultó inconsistente con el contenido real.	"Es una plaza muy amplia, excelente para cualqu..." • Real: Negativo

Evaluación de Robustez y Pruebas de Estrés

Se evaluó el comportamiento del modelo frente a 3 escenarios adversos reales (ruido ortográfico, textos extensos fuera de distribución y dialectos colombianos):

VIGILADA MINEDUCACIÓN

1. Prueba 1: Ruido Ortográfico y Elongación de Vocales
    - Entrada probada: "El sirvicio fue pésimooo y la atencion muuuuy lenta, no vuelvo jamás."
    - Predicción del Modelo: positivo (Confianza: 0.85)
    - Diagnóstico: Falla por falta de normalización. La repetición de caracteres (pésimooo, muuuuy) alteró la representación de subwords en el BPE del tokenizer de LLaMA, degradando la predicción.
2. Prueba 2: Texto Extenso (Truncamiento > 213 tokens)
    - Entrada probada: "El hotel es hermoso pero muy ruidoso muy ruidoso..." $(\times\ 20)$
    - Predicción del Modelo: neutral (Confianza: 0.95)
    - Diagnóstico: Efecto de Truncamiento. Al truncar la secuencia en max_length = 213, la parte final del texto (donde se concentraba la queja) fue omitida, clasificando el fragmento inicial como neutral.
3. Prueba 3: Regionalismos / Modismos Colombianos
    - Entrada probada: "La comida estuvo re bacana pero el servicio si una chanda total."
    - Predicción del Modelo: positivo (Confianza: 0.79)
    - Diagnóstico: Sesgo de Preentrenamiento. Palabras como "chanda" o "re bacana" no formaban parte del vocabulario estándar de TinyLlama, priorizando la probabilidad de palabras con carga positiva en el contexto general.

Aseguramiento de Calidad (QA) y Decisión de Negocio

Matriz de Calidad Aplicada en Fase 4 (CRISP-ML(Q))

- Riesgo Identificado: Sobreajuste al leaderboard público de Kaggle y alta variabilidad por desbalance de clases.
- Método QA Aplicado:
    1. Se empleó validación local estratificada con semilla fija (SEED = 42).
    2. Se evaluó la incertidumbre mediante Bootstrap (1,000 iteraciones).
    3. Se ejecutaron pruebas de estrés para medir la sensibilidad a textos adversos.
- Evidencia OBTENIDA: Reporte con $F1 - Macro = 0.7239$, 100 de precisión en predicciones negativas y 0 de confusiones de polaridad opuesta (Positivo ↔ Negativo).

Decisión Explícita frente a Criterios de Éxito

VIGILADA MINEDUCACIÓN

SI SE APROBÓ EL MODELO para avanzar a la Fase 5 (Despliegue) debido a los siguientes hallazgos:

1. Cumplimiento del Criterio ML: Se superó holgadamente el baseline no neuronal obligatorio (F1-Macro de 0.7239 frente a 0.6376 del baseline TF-IDF).
2. Eficiencia Técnica: El ajuste con QLoRA de 4-bit permitió ejecutar inferencias en menos de 0.05 segundos por reseña con un consumo de VRAM $< 2.7GB$, cumpliendo con las restricciones operativas.
3. Acción Mitigante Planteada para Producción: Se definió la inclusión de un módulo de normalización ligera en el pipeline de inferencia (Fase 5) para corregir caracteres repetidos antes de enviar el texto al tokenizer.

Fase 5. Despliegue

En esta fase se implementó un pipeline de inferencia modular, liviano y reproducible para la puesta en producción del modelo final (TinyLlama-1.1B + QLoRA). Se evaluaron los costos computacionales, la memoria utilizada, la latencia de inferencia y se construyó un componente de demo funcional para clasificar nuevas reseñas en tiempo real.

1. Pipeline de Inferencia y Generación de Envío (Kaggle)

Se construyó un script de inferencia independiente que realiza la carga de los adaptadores PEFT/LoRA entrenados en la Fase 3 y ejecuta la predicción por lotes sobre las 351 muestras del conjunto de prueba no etiquetado (test.csv).

Flujo del Pipeline:

- Inicialización y Tokenización: Carga del tokenizador preentrenado con padding_side = 'left' y token de relleno estandarizado (pad_token = eos_token).
- Carga en Memoria: Inicialización del modelo base en formato de precisión media (torch.float16) combinado con los pesos del adaptador LoRA guardados en el directorio de resultados del experimento final.
- Procesamiento por Lotes: Inferencia sobre el archivo test.csv utilizando un tamaño de lote de 16 para optimizar el paralelismo en la GPU sin riesgo de agotamiento de memoria VRAM.
- Post-procesamiento y Exportación: Mapeo de las etiquetas numéricas (0, 1, 2) a las clases textuales ('negativo', 'neutral', 'positivo') y generación del archivo submission.csv cumpliendo con los requisitos de la competencia.

VIGILADA MINEDUCACIÓN

2. Medición de Costos Computacionales y Eficiencia

Se midió el impacto operativo del modelo desplegado en términos de almacenamiento, consumo de memoria y tiempo de respuesta por ejemplo:

a. Métrica de Costos de Despliegue en Producción:
   - Tiempo de Carga del Modelo: 1.90 segundos (Carga ultrarrápida combinando la arquitectura base y el adaptador PEFT).
   - Peso del Adaptador LoRA en Disco: 12.08 MB (Extremadamente liviano al guardar únicamente las matrices de bajo rango actualizadas).
   - Consumo de Memoria RAM del Sistema: 2.98 GB (Bajo requerimiento de memoria volátil del sistema).
   - Consumo de VRAM GPU (Inferencia): 6.83 GB (Consumo estable que se mantiene dentro de los límites de la GPU T4 de Kaggle).
   - Tiempo Total de Inferencia en Test: 7.82 segundos (Procesamiento completo de las 351 reseñas del conjunto de prueba).
   - Latencia por Reseña (Tiempo/Ejemplo): 22.28 ms/reseña (Alta velocidad de respuesta adecuada para entornos de procesamiento por lotes o tiempo real).

3. Impacto de la Cuantización (4-bit QLoRA)

Como estrategia de compresión, se empleó cuantización NormalFloat4 (NF4) mediante la librería bitsandbytes durante las Fases 3 y 5.

- Ahorro de Memoria: El modelo base original requeriría aproximadamente 4.4 GB de VRAM solo para cargar sus pesos en precisión completa. Con la cuantización a 4 bits combinada con LoRA, se logró ejecutar la inferencia de manera fluida utilizando solo 12.08 MB adicionales para el adaptador.
- Impacto en la Métrica: La cuantización no degradó el rendimiento del modelo en validación local, permitiendo mantener el F1-Macro en 0.7239, superando ampliamente al baseline no neuronal de 0.6376.

4. Demo Mínima Funcional

Se implementó una función de inferencia interactiva clasificar_reseña(texto) que simula la integración en un sistema de recepción de reseñas turísticas en tiempo real.

Ejemplo de Ejecución:

- Entrada de Texto: "La atención en el restaurante fue excelente, los platos deliciosos y la vista increíble."
- Resultado del Sistema:
  - Sentimiento Predicho: neutral
  - Nivel de Confianza: 99.27%
  - Desglose de Probabilidades: negativo: 0.6%, neutral: 99.3%, positivo: 0.2%

VIGILADA MINEDUCACIÓN

Nota de análisis sobre la demo: El modelo clasificó el texto como 'neutral' con alta confianza (99.27%). Esto refleja la tendencia conservadora del modelo aprendida durante el entrenamiento, debido a que el 78.1% de los datos de entrenamiento correspondían a la clase neutral, lo que lleva al modelo a priorizar dicha clase en textos con adjetivos descriptivos generales.

Esta demo validó la capacidad del pipeline para procesar entradas en tiempo real y entregar un desglose de probabilidades mediante una función simple lista para integrarse con interfaces como Gradio o APIs REST.

Fase 6. Monitoreo y mantenimiento

En esta fase se definió un plan continuo de monitoreo y mantenimiento para el modelo desplegado en producción, asegurando la detección temprana de degradación en el desempeño, cambios en la distribución de datos (data drift) y sesgos conceptuales (concept drift).

1. Variables de Monitoreo Continuo

Para garantizar el correcto funcionamiento del sistema en un entorno real, se estructuró un esquema de observabilidad centrado en cuatro métricas clave:

- Distribución de Clases Predichas: Monitoreo semanal de la proporción de predicciones ('negativo', 'neutral', 'positivo'). Un desvío superior al 15% respecto a la distribución de referencia (78% neutral, 19% positivo, 3% negativo) servirá como alerta de drift en las reseñas entrantes.
- Nivel Promedio de Confianza: Seguimiento del valor de probabilidad máximo retornado por la función softmax. Un descenso en la confianza media por debajo del 75% indicará que el modelo está procesando textos fuera de su dominio de entrenamiento.
- Longitud en Tokens de los Comentarios: Evaluación de la longitud media de los textos recibidos. Comentarios que superen consistentemente el max_length fijado (213 tokens) alertarán sobre la necesidad de ajustar la estrategia de truncamiento.
- Tasa de Vocabulario Nuevo (Out-of-Vocabulary / Unk Tokens): Conteo de palabras no reconocidas o fragmentadas en exceso por el tokenizer BPE de LLaMA, señalando la aparición de nuevos regionalismos, neologismos o modismos turísticos.

2. Simulación de Drift y Cuantificación de Degradación

Se simuló un escenario severo de cambio de distribución evaluando el modelo sobre un subconjunto de control compuesto por reseñas que incorporaban ruido ortográfico, elongación de caracteres, expresiones informales y modismos regionales colombianos (como "re malo", "una chanda", "una chimba", "dio piedra").

Resultados Obtenidos en la Simulación:

VIGILADA MINEDUCACIÓN

- F1-Macro en Validación Original: 0.7239
- F1-Macro en Subconjunto con Drift: 0.2500
- Degradación Cuantificada del Desempeño: 65.46%
- Confianza Promedio del Modelo: 71.85%

Análisis Causal de la Degradación: La simulación evidenció una caída drástica del 65.46% en la métrica oficial F1-Macro. Se observó que el modelo tendió a clasificar erróneamente las críticas negativas informales como reseñas 'positivas' (por ejemplo, "El sirvicio fue re malo..." clasificó positivo con 93.02% de confianza, y "Pésimooo todo..." clasificó positivo con 78.71% de confianza). Este comportamiento se debió a que las secuencias de subwords alteradas por errores tipográficos y los modismos no estándar activaron sesgos hacia la clase mayoritaria o confundieron las representaciones internas de la atención, demostrando la alta vulnerabilidad del modelo ante textos fuera de distribución sin un preprocesamiento de normalización.

3. Criterios de Reentrenamiento y Roles Responsables

Con base en la alta degradación observada en la simulación, se establecieron los umbrales operativos que activarán el reentrenamiento del modelo y se asignaron las responsabilidades del equipo según el marco CRISP-ML(Q):

Criterios de Activación para Reentrenamiento:

- Degradación del F1-Macro por debajo de 0.55 en las evaluaciones periódicas sobre datos etiquetados de producción.
- Caída de la confianza media global por debajo del 70% durante tres días consecutivos.
- Acumulación de más de 300 nuevas reseñas informales/regionales etiquetadas y validadas manualmente por el equipo.
- Detección de una tasa de errores tipo 'falso positivo' superior al 20% en auditorías semanales de calidad.

Asignación de Responsabilidades:

- Ingeniero de Datos: Responsable de implementar un módulo previo de normalización ortográfica en el pipeline, monitorear la distribución de tokens y limpiar los nuevos datos recolectados.
- Ingeniero de Modelado / ML: Responsable de ejecutar el reentrenamiento del adaptador LoRA incorporando datos de aumentación con ruido dialectal y ajustar los hiperparámetros.
- Responsable de Evaluación y QA: Encargado de la revisión manual de los casos con baja confianza, validación de la matriz de confusión posterior al reentrenamiento y aprobación final del nuevo checkpoint para producción.

VIGILADA MINEDUCACIÓN

Conclusiones

1. Cumplimiento de Criterios de Éxito de Machine Learning: El modelo final basado en la arquitectura Transformer TinyLlama-1.1B ajustado mediante QLoRA (EXP-05) alcanzó un F1-Macro en validación local de 0.7239 y una Accuracy global de 91.46%. Este desempeño superó con un margen amplio al baseline no neuronal obligatorio (TF-IDF + Regresión Logística, que obtuvo un F1-Macro de 0.6376), demostrando la superioridad de la representación de lenguaje mediante atención frente a los enfoques tradicionales de bolsa de palabras.
2. Eficiencia y Viabilidad Operativa: Se demostró que la técnica de cuantización en 4 bits (NF4) combinada con adaptadores LoRA permite entrenar y desplegar un modelo de más de mil millones de parámetros utilizando menos de 2.7 GB de VRAM en GPU. Con un tamaño de archivo de adaptador de solo 12.08 MB y una latencia por ejemplo de 22.28 ms en inferencia, el sistema cumple con los criterios de éxito económico y técnico establecidos para la competencia.
3. Evaluación de Calidad y Criterio de Negocio: A pesar del desbalance extremo del dataset (78.1% clase neutral), el modelo logró un 100% de precisión en la clase negativa, garantizando que el sistema no emita falsos alarmas de críticas negativas cuando un usuario está satisfecho. Esto respalda la decisión explícita de aprobar el modelo para apoyar la toma de decisiones de operadores turísticos regionales.

Limitaciones y Trabajo Futuro

1. Sensibilidad a Texto Fuera de Distribución (Data Drift): La prueba de estrés en la Fase 6 reveló una degradación del 65.46% en el F1-Macro frente a reseñas con errores tipográficos, elongación de vocales y modismos regionales colombianos. Como trabajo futuro, se proyecta integrar una etapa previa de normalización de texto e incluir técnicas de data augmentation (como sustitución de sinónimos dialectales y ruido sintáctico) durante el reentrenamiento.
2. Tamaño Muestral de la Clase Minoritaria: La baja presencia de ejemplos negativos en el conjunto de entrenamiento (solo 22 registros de 818) restringió el recall de dicha clase (0.2500). En iteraciones futuras se propone explorar técnicas de re-muestreo guiado o el uso de funciones de pérdida focal (Focal Loss) para penalizar con mayor fuerza los errores en clases minoritarias.
3. Ambigüedad en Reseñas Mixtas: El modelo actual tiende a clasificar comentarios con opiniones contrapuestas (por ejemplo, aspectos positivos de la infraestructura pero negativos del servicio) dentro de la clase neutral o mayoritaria. Se plantea explorar cabezas de clasificación multi-etiqueta o análisis de sentimientos basado en aspectos (ABSA).

Declaración de Uso de IA Generativa

De acuerdo con los lineamientos de la guía del proyecto (Sección 4), se declara que se utilizaron herramientas de Inteligencia Artificial Generativa (asistentes de LLM) durante el desarrollo del proyecto bajo el siguiente esquema:

VIGILADA MINEDUCACIÓN

- Propósito de Uso: Asistencia en la depuración de código Python/PyTorch para el pipeline de Hugging Face y PEFT, estructuración de sintaxis para gráficos de Seaborn/Matplotlib, y apoyo en la redacción y organización de las secciones del informe técnico según la metodología CRISP-ML(Q).
- Validación y Control Humano: Todo el código generado por la IA fue revisado, adaptado y probado directamente en el entorno de Kaggle por los integrantes del equipo. Las métricas, tablas de experimentos, análisis de errores y conclusiones fueron verificadas contra las salidas reales del modelo, corrigiendo alucinaciones de código o interpretaciones teóricas imprecisas.

Contribución Individual de cada Integrante

A continuación se detalla la distribución de roles y la contribución individual de cada miembro del grupo durante el ciclo de vida del proyecto:

Tabla de Contribución Individual:

Integrante	Rol Principal	Contribuciones Clave Realizadas
[Nombre del Integrante 1]	Datos y Modelado	Descarga y verificación del dataset, análisis exploratorio (EDA), tokenización con LLaMA Tokenizer, prevención de fuga de datos, implementación de la grilla de experimentos QLoRA (EXP-00 a EXP-05) y ajuste de hiperparámetros (Fases 1, 2 y 3).
[Nombre del Integrante 2]	Evaluación, Despliegue y Documentación	Ejecución del pipeline de evaluación cuantitativa y cualitativa en Fase 4, análisis de los 14 errores, pruebas de robustez, implementación del pipeline de inferencia (submission.csv), simulación de drift (Fases 5 y 6) y redacción del informe técnico.

VIGILADA MINEDUCACIÓN